

Does per-point color help point-cloud completion?

The data-scale curve

Weekly research report 2026-07-28 — 21–28 July 2026

Efe Yenice (researcher) · Claude (AI co-researcher)

Question of the week: does the geometry penalty from per-point color input shrink—or flip—as training data grows?

TL;DR — the week in three findings

1. **The color-input harm shrinks with data, but does not flip.** Feeding per-point RGB into the completion network still hurts geometry at every tested scale: the C_colin-vs-A_geo test CD-L1 gap falls from 6.6 % worse at $N=20$ models/category to 4.8 % worse at $N=60$ and 4.0 % worse at $N=150$ (mean of 3 seeds; every seed agrees on the sign: 3.7–4.3 %). The trend supports the small-data-artifact hypothesis, but no crossover has been observed yet (Figure 1).
2. **Predicting color stays free — the cost is the input, not the output.** B_joint (which also *outputs* color) matches or slightly beats C_colin on geometry at every rung (3.7 % worse vs. 4.0 % worse at $N=150$). Its learned color head now beats the NN-copy (0.073 vs. 0.082 MSE) and median-copy baselines, but still loses to the trivial *mean-copy* baseline (0.070); color prediction quality is the next open gap (Figure 2).
3. **The pipeline is now disconnect-proof, and that is what made the week possible:** 600 freshly generated model bundles and 15 training runs (1595 epochs total) streamed to Hugging Face with per-unit atomic commits and idempotent resume; Colab disconnects and VM recycles cost at most one unit of work.

1 The story so far

The project asks one controlled question: *does adding per-point RGB color improve point-cloud completion, and under what conditions?* The cross-modal literature says color helps a lot — but in a different setting: methods like EGInet inject a separate, complete *guidance image* ($\sim 58\%$ CD reduction over geometry-only baselines). Our setting is stricter and unstudied: the color rides *on the partial points themselves* (and, in one arm, must also be *predicted* for the completed cloud).

To isolate color’s effect we train one from-scratch PCN backbone in three arms that differ only in their input/output channels:

arm	input	output	role
A_geo	xyz	xyz	geometry baseline
B_joint	xyzrgb	xyzrgb	does <i>outputting</i> color cost geometry?
C_colin	xyzrgb	xyz	does color <i>input</i> help geometry?

The first controlled result (previous report, ~ 15 models/category) was the opposite of the literature: color input *hurt* geometry by $\sim 17\%$, most plausibly because at tiny scale per-point color acts as a per-model fingerprint that eases memorization instead of generalization. That made **data scale the binding constraint**, and this week’s question follows directly: *track the harm as a function of training-set size N* — if it is a small-data artifact, it should shrink, and eventually flip (ADR-0007).

2 What we did this week

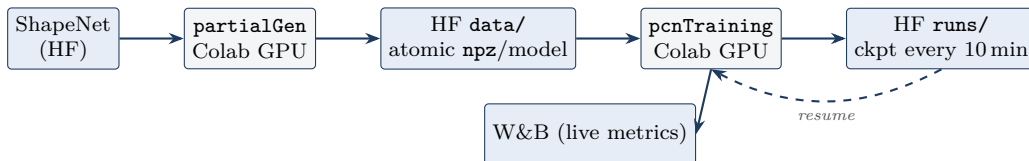
2.1 The experiment: a data-scale curve

Train all three arms at $N \in \{20, 60, 150\}$ models per category (airplane, car, chair) and read the C-vs-A gap as a function of N . Design choices that make the curve trustworthy:

- **One variable.** A *fixed held-out test set* (30 models/category, never trained on, reserved before any training) is scored at every rung, and the training pools are *nested* ($N=20 \subset 60 \subset 150$), so N is the only thing that changes along the curve. A fixed validation set (15/category) drives checkpoint selection and early stopping (patience 25 epochs, validated every 10, budget 300).
- **Seeds where they matter.** The top rung — where the claim lives — runs 3 seeds per arm; lower rungs run 1 (ADR-0007’s cost/precision trade).
- **Baselines that give color numbers meaning.** Three training-free color-copy predictors (NN-copy, mean-copy, median-copy from the partial input) plus the NN-*oracle* floor bracket the learned color head from below and above.
- **Same generation for every model.** 600 bundles regenerated from scratch with the colored sampling pipeline (dense EPFL colored cloud $\rightarrow$ color-aware 8192-point GT $\rightarrow$ 8 seeded `meshray_nn` partial views; 4 views per model used in training). Fully textureless models are dropped (color std ≤ 0.01), leaving 433 of 450 pool models at the top rung.

2.2 The ops overhaul that made it runnable

Last week’s runs kept dying with Colab disconnects: results lived in Python lists, intermediates on the VM’s ephemeral disk, and a death at 99 % cost 100 %. This week we rebuilt the operational layer around one rule (ADR-0008): **“done” means bytes on Hugging Face, never “in RAM”** — every long job commits per unit and resumes idempotently.



Concretely: the data generator writes one atomic `{model_id}.npz` per model and a background scheduler streams it to the HF dataset (a crash costs one model; re-running the cell skips everything already uploaded); training streams checkpoints and `metrics.csv` to HF every 10 minutes and logs live to W&B; the curve driver resumes mid-run after a VM recycle by reloading prior state from HF. Code lives in git only (notebooks output-stripped); Google Drive is fully retired. The week’s runs were completed across multiple Colab sessions with no lost work.

3 Results

3.1 The headline: harm vs. scale

Figure 1 and Table 1 carry the week’s answer. All arms improve steeply with data ($23.68 \rightarrow 18.56$ CD-L1 for `A_geo` alone — data remains the dominant lever for everything). Within-rung, the ordering never changes: `A_geo` best, both color-input arms behind. The gap narrows with every step of N — consistent with the memorization-fingerprint account — but at $N=150$ /category it is still 4.0 % worse, and *every* top-rung seed pairing agrees on the sign (3.7, 4.1, 4.3 %). F-score tells the same story (0.46 vs. 0.44 at $N=150$). We therefore report: **the harm behaves like a small-data artifact, but has not vanished at the scales we can currently reach**. No extrapolated crossover is claimed from three points.

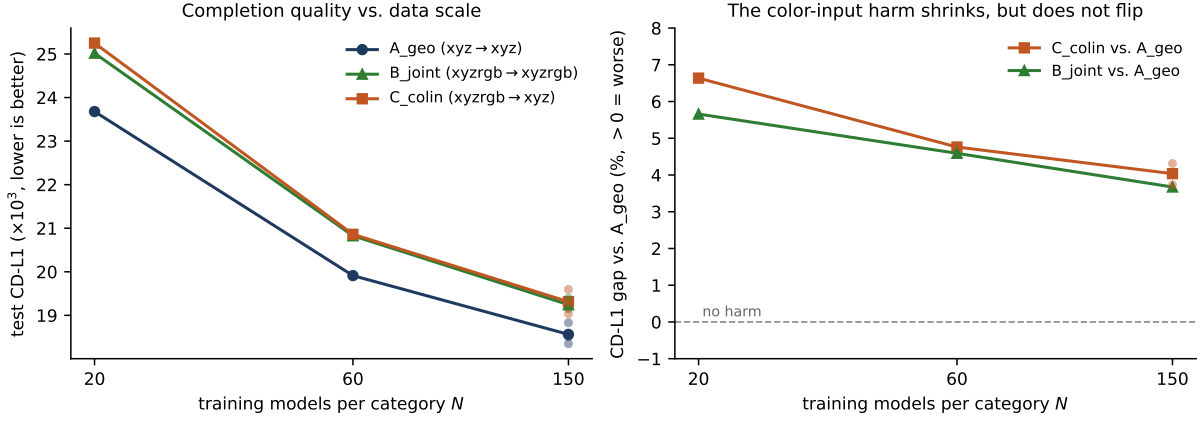


Figure 1: The data-scale curve on the fixed held-out test set (90 shapes). Left: absolute test CD-L1 per arm vs. training models per category N ; faint points are the individual seeds at $N=150$. Right: the same data as a gap relative to A_geo ; positive means color made geometry worse. The harm shrinks monotonically (6.6 % worse $\rightarrow$ 4.8 % worse $\rightarrow$ 4.0 % worse) but stays positive at every tested scale.

N /cat	train models	A_geo	C_colin	B_joint	C vs. A	B vs. A	B color MSE
20	60	23.68	25.25	25.02	+6.6 %	+5.7 %	0.085
60	180	19.91	20.86	20.83	+4.8 %	+4.6 %	0.078
150	433	18.56	19.31	19.24	+4.0 %	+3.7 %	0.073

Table 1: Test CD-L1 ($\times 10^3$, lower is better; $N=150$ row averages 3 seeds) and the gaps relative to the geometry baseline. Copy-baseline color MSEs for comparison: NN 0.082, median 0.081, mean 0.070; oracle floor 0.020. Full per-run table in Section A.

3.2 The color head vs. its baselines

Two honest reads. First, the *geometry* cost of adding the color *output* is zero: B_joint lands at or slightly ahead of C_colin at every rung (Table 1), so predicting color is not what hurts — ingesting it is. Second, the color the head predicts is still weak: it has learned enough to beat copying colors from the partial (NN-copy, 0.082) but not enough to beat answering “the average color of this object” (mean-copy, 0.070 vs. learned 0.073 at $N=150$). The qualitative panels (Figure 3, right column) show why: under an MSE-reported, BCE-trained objective the head hedges toward washed-out, low-saturation colors. Improving this head — including the advisor’s Gaussian-distribution ideas — is a concrete next thread, and the copy baselines now give it a meaningful bar to clear.

3.3 Qualitative check

Figure 3 keeps us honest about absolute quality: at $N=150$ /category the completions capture the coarse shape but none of the fine structure. That is expected — the PCN literature trains on $\sim 30,000$ models, two orders of magnitude more than our largest rung — and it is why every claim in this report is *relative*, *within-pipeline*: same data, same splits, same budget, arms differing only in color channels. The training curves (Section B) show the same thing quantitatively: in all 15 runs validation plateaus early while train keeps improving — the classic data-limited signature, at every rung we can afford.

3.4 Continuity with the previous result

The prior report measured the harm at $\sim 17\%$ (~ 15 models/category); this pipeline’s smallest rung ($N=20$) measures 6.6 % worse. The sign and the “harm at small N ” conclusion replicate; the

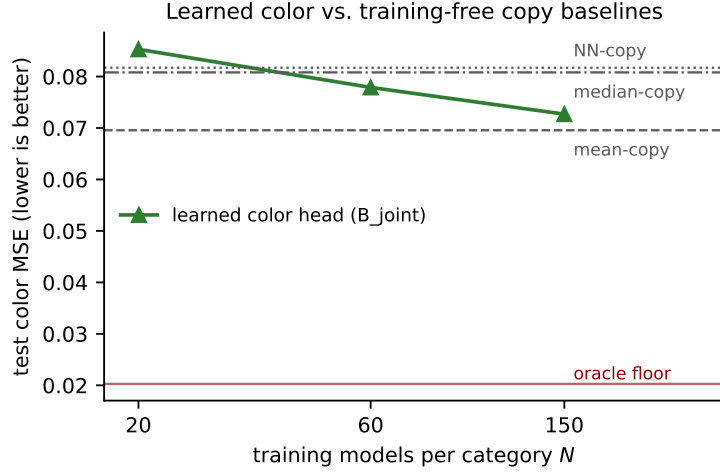


Figure 2: Learned color vs. training-free baselines (test color MSE at the GT’s nearest predicted point). The learned head improves with data and has passed NN-copy and median-copy, but not yet the mean-copy baseline; the oracle floor (0.020) shows most of the headroom is still unclaimed.

magnitude does not, and we flag that plainly. Between the two measurements the pipeline changed in ways that all plausibly shrink the measured gap: a fixed, larger held-out test set (30/category vs. ad-hoc); early stopping on validation (the old fixed-epoch training let arms overfit for different lengths); one partial-generation method (`meshray_nn`) instead of a mix; 4 training views per model instead of 8; and freshly regenerated data. This is exactly why the curve was designed to hold everything but N fixed — cross-pipeline magnitudes are not comparable, within-pipeline trends are. The $\sim 17\%$ number should be treated as superseded by this curve.

4 Honest read

- **What we can claim:** within a controlled, single-variable design, the color-input geometry penalty decreases monotonically with training-set size (6.6 % worse $\rightarrow$ 4.0 % worse) and the color-output cost is ≈ 0 . This is consistent with the small-data-artifact hypothesis and is the first scale-resolved measurement of the effect in the per-point-color setting.
- **What we cannot claim yet:** that the harm vanishes or flips. Three points, one backbone (PCN), three categories, modest scale. The residual 4.0 % worse at $N=150$ is real (sign-consistent across seeds) but small relative to seed spread in absolute CD (A’s three seeds span 18.35–18.83); only the matched-seed, same-split design makes it readable at all.
- **Alternative explanations still open:** the harm could persist at any feasible N for this backbone (PCN’s global bottleneck may simply waste capacity on color); or flip only with architectures that fuse color later/differently. Both are testable (below).
- **Color prediction is behind color ingestion:** the learned head has not beaten mean-copy. Any future “color helps” story needs the output side to clear its trivial baselines first.

5 Proposed next steps

In recommended order (1–2 are one week’s spine; the advisor meeting decides):

1. **Widen the curve: more categories.** Add 3–5 ShapeNet categories at fixed N (the advisor’s own suggestion). Tests whether the shrinking-harm trend is category-general, and grows total training data — the strongest known lever — without new machinery.
2. **Firm the curve: seeds for the lower rungs.** 3 seeds at $N=20$ and $N=60$ (~ 6 cheap runs) put error bars on the whole curve, not just its top.

One held-out test shape at the top rung ($N = 150$): three input views, all three arms

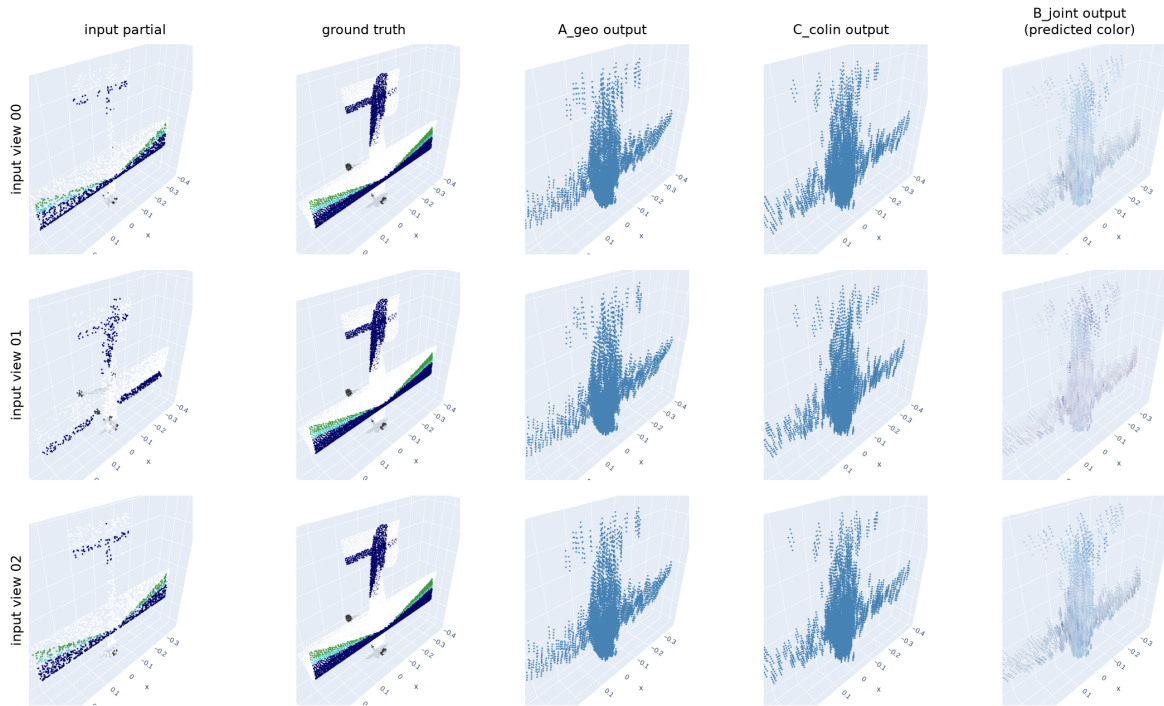


Figure 3: One held-out test airplane at $N=150$, three input views (rows), recomposed from the run’s own renders. Columns: input partial, colored GT, and each arm’s fine output (8192 points; B_joint’s shown in its predicted colors). Completions are recognizable but coarse at this scale, and B’s predicted colors are visibly desaturated — the regression-to-the-mean behavior behind Figure 2.

3. **Extend the curve:** $N > 150$. Airplane/car/chair have thousands of models each; data generation is the only cost (the resilient pipeline makes it a background job).
4. **The color head:** attack the mean-copy gap — the advisor’s Gaussian output-distribution idea, or NN/mean/median-aware hybrid heads, now with honest baselines to beat.
5. **Second backbone:** SnowflakeNet or AdaPoinTr as arm-for-arm replication, testing whether the harm is PCN-specific (the advisor’s “PCN vs. Snowflake” comparison).
6. **Demo Space:** an interactive HF Space over the trained checkpoints for advisor-facing inspection (checkpoints are already public).

How this week was run

Roles. Efe: direction, decisions at forks, and the Colab runs. Claude (AI co-researcher): experiment design, all code, local testing, run analysis, documentation, and this report. **The loop** (documented in docs/WORKFLOW.md): decide → grill the plan → ADR → build + test locally → run resiliently on Colab → analyze → document + report. **This week in numbers:** 9 merged PRs (#8–#16), 2 ADRs (0007 scale curve, 0008 research ops), 600 model bundles regenerated, 15 training runs (1595 epochs, early-stopped between 65 and 145), zero lost runs after the resilience rewrite. Full timeline and incidents: Section D.

A All 15 runs

run	arm	N/cat	seed	CD-L1	CD-L2	F@1%	color MSE	best ep	ep trained
n020_A_geo_s1234	A_geo	20	1234	23.68	1.387	0.350	–	59	85
n020_B_joint_s1234	B_joint	20	1234	25.02	1.496	0.329	0.085	69	95
n020_C_colin_s1234	C_colin	20	1234	25.25	1.511	0.323	–	79	105
n060_A_geo_s1234	A_geo	60	1234	19.91	1.014	0.424	–	99	125
n060_B_joint_s1234	B_joint	60	1234	20.83	1.094	0.404	0.078	109	135
n060_C_colin_s1234	C_colin	60	1234	20.86	1.078	0.402	–	99	125
n150_A_geo_s2	A_geo	150	2	18.51	0.955	0.468	–	119	145
n150_A_geo_s3	A_geo	150	3	18.83	0.967	0.457	–	59	85
n150_A_geo_s1234	A_geo	150	1234	18.35	0.926	0.467	–	89	115
n150_B_joint_s2	B_joint	150	2	19.19	0.995	0.450	0.073	99	125
n150_B_joint_s3	B_joint	150	3	19.39	0.997	0.445	0.072	69	95
n150_B_joint_s1234	B_joint	150	1234	19.15	0.969	0.440	0.073	99	125
n150_C_colin_s2	C_colin	150	2	19.31	0.987	0.446	–	39	65
n150_C_colin_s3	C_colin	150	3	19.60	1.018	0.443	–	69	95
n150_C_colin_s1234	C_colin	150	1234	19.03	0.952	0.443	–	49	75

CD $\times 10^3$ in the near-unit `model_normalized` frame; CD-L1 = symmetric mean nearest-neighbor distance (avg. of both directions), CD-L2 its squared counterpart; F@1 % at $\tau=0.01$; color MSE in $\text{RGB} \in [0, 1]^3$ at the GT’s nearest predicted point. “best ep” = checkpoint kept (lowest val CD-L1); “ep trained” includes the 25-epoch patience tail.

B Training curves

All 15 runs: train/val CD-L1 per epoch (dashed line = best epoch kept by early stopping)

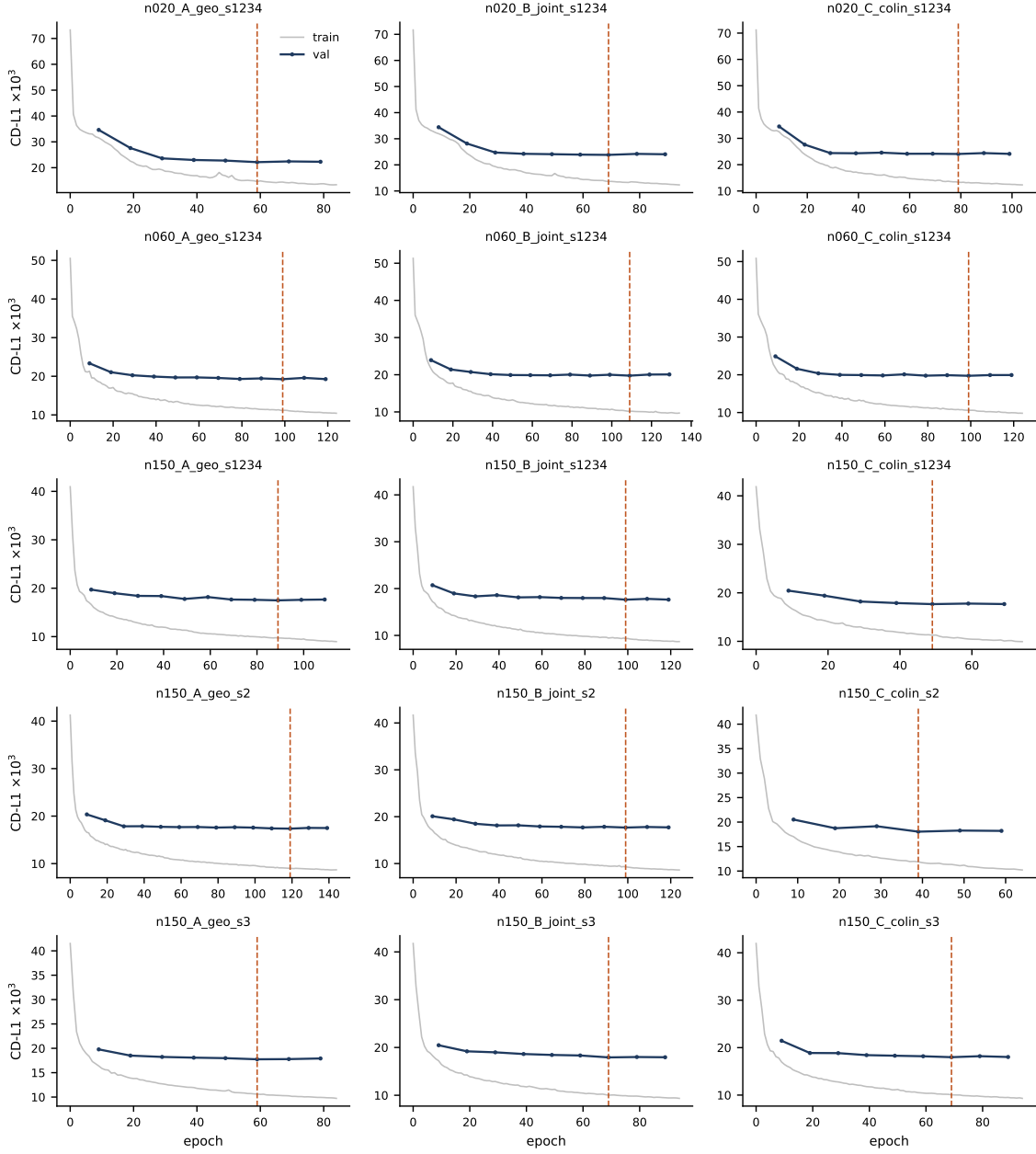


Figure 4: All 15 runs: train CD-L1 (gray) vs. validation CD-L1 (navy, every 10 epochs); the dashed line marks the best epoch whose checkpoint was kept. Validation plateaus early in every run while train keeps falling — the data-limited signature motivating next week’s proposals.

C Provenance & how to reproduce

- **Code:** <https://github.com/efeyenice/point-cloud-completion-research> — the experiments ran from `main` at commit `ffa7bb3` (the #8–#16 series). Model, splits, and drivers: `notebooks/pcnTraining.ipynb`. Data generation: `notebooks/partialGeneration.ipynb` + `tools/pc_resilient.py`.
- **Data + run artifacts:** HF dataset [efeyenice/pc-completion-data](https://huggingface.co/datasets/efeyenice/pc-completion-data) at revision `0db547ff6b`: `data/` (600 npz bundles: 8192-point colored GT + 8 partial views + metadata) and `runs/`

(per-run `config.json`, `metrics.csv`, best/latest checkpoints, `curve.csv`, panels).

- **Live metrics:** W&B project `pc-completion`, entity `efeyenice004-sabanc-university-nanotechnology-research-a`.
- **Model & budget (from each run's `config.json`):** from-scratch PCN encoder-decoder, input 2048 points, coarse 1024, fine 8192; Adam, lr 10^{-4} , step decay $\times 0.7$ every 50 epochs; batch 16; budget 300 epochs, early stop patience 25 (val every 10); split seed 42; train seeds {1234, 2, 3}.
- **Reproduce:** open `pcnTraining.ipynb` on Colab (GPU), add `HF_TOKEN` + `WANDB_API_KEY` to Secrets, run T0–T7 then T9 (the curve driver skips runs already on HF and resumes partial ones — re-running is always safe), then T10 for figures/panels. This report: **make figures** && **make** in `reports/2026-07-28-scale-curve/` — every number in the prose is regenerated from the HF artifacts.

D Process log — the week as it actually happened

Timeline

- **Jul 21–22.** Advisor meeting notes (compare architectures; explore color; more data; report numbers + visuals) distilled — via a grilling session — into one spine: the data-scale curve (ADR-0007). Harness built and locally validated: GPU FPS for partial generation (~ 8 min $\rightarrow \sim 5$ s/model, numpy fallback bit-identical), fixed test/val + nested train pools (unit-tested), color-copy baselines, early stopping, curve driver + report cells. PR #8.
- **Jul 27 (morning).** Recurring pain — Colab disconnects killing long runs and a growing Drive dependency — triggered the research-ops decision (ADR-0008): git for code, HF for data + run artifacts, W&B for metrics, Drive retired. Built and merged: resilient per-model data-gen module (#10), HF-to-training loader (#11), training wired to HF + W&B (#12), repo restructured for adopters + the co-research workflow written up (#9, #13, #14).
- **Jul 27 (midday).** Data generation on Colab: 600 bundles streamed to HF. Incidents: a `NameError` (`NAME_BY_SYNSET` lived in a cell the run recipe skipped) fixed forward as #15; one model failed mid-run — the driver logged it, kept going, and the re-run picked it up (that is the resilience layer working as designed).
- **Jul 27 (afternoon).** Training gates passed on the new data; curve runs started. Mid-curve, the remaining single point of failure surfaced — a VM recycle would have restarted the curve — so the driver was made resumable from HF state (#16) *while the curve was running*, and the run continued across sessions.
- **Jul 27–28.** Top rung (9 runs) completed; `curve.csv` + panels landed on HF; this report built.

Incidents $\rightarrow$ lessons

1. **Runs died with the VM.** Lesson (now policy, ADR-0008): per-unit atomic commits to durable storage + idempotent resume, for *every* long job. “Done” means bytes on HF.
2. **A skipped cell held a needed name.** Notebook cells form a hidden dependency graph; any cell a recipe skips must not define names others use. Fixed structurally in #15.
3. **Keep-alive $\neq$ resilience.** Headless/keep-alive running (`colab-cli`) reduces disconnects but cannot prevent VM eviction — it complements, never replaces, resumability (#16).
4. **Secrets hygiene.** API keys were pasted into a chat transcript during setup; they were treated as compromised and rotated, and all notebooks read secrets from Colab Secrets/env only. Keys never enter git, notebooks, or reports.

E Artifact map

artifact	home
code + notebooks (source of truth)	git — <code>efeyenice/point-cloud-completion-research</code>
decisions + rationale	git — <code>docs/adr/</code> (0001–0008), <code>CONTEXT.md</code> , <code>notes/</code>
method (how we co-research)	git — <code>docs/WORKFLOW.md</code>
weekly reports	git — <code>reports/</code> (this document)
generated datasets	HF dataset <code>efeyenice/pc-completion-data</code> (<code>data/</code>)
run artifacts (ckpts, metrics, curve)	same HF dataset (<code>runs/</code>)
live training metrics	W&B project <code>pc-completion</code>
demo (planned)	HF Space <code>efeyenice/pc-completion-runs</code>
scratch / intermediates	Colab local disk (ephemeral by design)